

NUTZUNGSBEDINGUNGEN

=====

Dokumentstand: Aktiver Proof-of-Concept und Entwicklungsstand.

DevBox besitzt einen funktionsfähigen lokalen Launcher, eine grafische Hauptoberfläche, modulare Strukturansichten, Stammdaten- und Dokumentationspflege, Dokumentations-Snapshots, lokale Werkzeugerkennung, eine Repository-Seite sowie erste Bausteine für den DevBox-spezifischen Veröffentlichungsprozess.

Der deskNode-Bereich ist aktiv in die DevBox integriert. Supervisor und Daemon können aus der GUI gestartet und gestoppt werden; deren Konsolenausgabe wird im deskNode-Log angezeigt. Die Produktversion kann aus den Stammdaten bearbeitet werden. UX-Themes können benannt, angelegt, gelöscht, dupliziert, umbenannt und gespeichert werden. Nach dem Speichern wird die deskNode-Manufakturdatenbank aktualisiert.

Die Symbolverwaltung ist als erste funktionsfähige Katalogoberfläche vorhanden. Sie verwaltet deskNode-Gerätekategorien und Verbrauchergeräte über Auswahlmenüs sowie Dialoge zum Anlegen, Bearbeiten und Löschen. Neue Geräte erhalten eine einzelne PNG-Quelle, die unter ihrer record_id als symbol_source_<record_id>.png abgelegt wird. Nach jeder erfolgreichen Katalogänderung wird create_manufacturer_db.py ausgeführt, damit mnfctr_db.r0b die aktuellen Theme-, Kategorien- und Gerätedaten enthält.

Der Graphic-Pack-Build ist als Proof-of-Concept funktionsfähig. Er erzeugt aus Symbolquellen und UX-Themes mehrere vorbereitete Grafikzustände für Verbraucher. Die eigentliche deskNode-Laufzeitintegration, Asset-Auswahl, Sprachpakete und vollständige Zustandslogik werden noch weiterentwickelt.

DevBox ist keine fertige Endnutzer- oder Release-Version. Oberflächen, Datenbankmigrationen, Produktmodule, Veröffentlichungsabläufe, Dokumentvorlagen und Tests werden fortlaufend überarbeitet. Diese Dokumentation beschreibt den derzeitigen Stand und muss bei funktionalen Änderungen mitgepflegt werden.

Erstveröffentlichung: Autor / Herausgeber: Markus Walloner Land: Germany (DE)

1. Gegenstand

Diese Nutzungsbedingungen beziehen sich auf die in der begleitenden Projektdokumentation beschriebene Software bzw. das beschriebene Projekt.

Kurzbeschreibung: DevBox ist die lokale Entwicklungszentrale der CYXnTrol Development Platform. Die PySide6-Anwendung bündelt Projektstruktur, Stammdaten, Dokumentation, lokale Werkzeuge, Produktmodule, Repository-Vorbereitung und wiederkehrende Entwicklungsabläufe.

DevBox ist kein Endnutzerprodukt. Es ist eine interne Arbeitsumgebung, in der technische Anforderungen, Datenmodelle, Oberflächen, Build-Schritte und Veröffentlichungsstände nachvollziehbar vorbereitet, geprüft und weiterentwickelt werden.

Langbeschreibung: DevBox verbindet Aufgaben, die sonst über Einzelskripte, Ordner, Tabellen, Konsolenfenster und externe Programme verteilt wären. Die Anwendung verwaltet zentrale Projekt- und Produktdaten, stellt Dokumentationsinhalte bereit, organisiert globale Strukturvorlagen, erkennt lokale Werkzeuge und bündelt produktbezogene Entwicklungsfunktionen in einer gemeinsamen Oberfläche.

Ein aktives Produktmodul ist deskNode. Die deskNode-Seite kann den lokalen Supervisor und Daemon starten und stoppen, die Produktversion aus den Stammdaten bearbeiten, UX-Themes pflegen und die Symbolverwaltung öffnen. Themes werden mit benannten Datensätzen, RGBA-Farben, globalen Schriftdateien, Größen, Konturen und Formregeln gepflegt. Nach jeder erfolgreichen Theme-Änderung wird die produktnahe deskNode-Manufakturdatenbank aktualisiert.

Die Symbolverwaltung ist die zentrale Quelle für Gerätekategorien und Verbrauchersymbole. Kategorien und Geräte werden in der DevBox-Datenbank angelegt, bearbeitet oder gelöscht. Beim Anlegen eines neuen Verbrauchergeräts wird genau eine PNG-Quelle angenommen und anhand der stabilen Datensatz-ID als `symbol_source_<record_id>.png` im globalen Grafikordner abgelegt. Der Graphic-Pack-Build erzeugt daraus vorberechnete Varianten für Themes und Zustände, damit die spätere deskNode-Laufzeit nur passende Assets auswählen muss.

DevBox arbeitet lokal im Projektkontext. Ein Launcher ermittelt den Projektroot über die Datei `.root`, stellt Laufzeitdaten unter AppData bereit, prüft externe Werkzeuge und startet die GUI aus einer temporären Arbeitskopie. Der echte Projektroot bleibt die maßgebliche Quelle für Ressourcen, Datenbanken und Funktionsskripte. Eine Einzelinstanzlogik verhindert parallele DevBox-Fenster und aktiviert beim erneuten Start die bereits geöffnete Instanz.

Das Projekt entsteht in einem KI-gestützten, iterativen Entwicklungsprozess. Fachliche Anforderungen, Prozesslogik, Datenmodelle, UX-Entscheidungen, Testfälle und Abnahmen werden durch den Projektverantwortlichen gesteuert; Implementierungsschritte werden mit KI-gestützten Werkzeugen erstellt, überprüft und fortlaufend nachgeschärft.

2. Nutzungsbedingungen

NUTZUNGSBEDINGUNGEN – ENTWURFSFASSUNG

1. Geltungsbereich

Diese Nutzungsbedingungen beziehen sich auf DevBox, die lokale Entwicklungsumgebung der CYXnTrol Development Platform. DevBox ist ein Entwicklungswerkzeug und keine fertige produktive Endnutzeranwendung.

2. Entwicklungsstatus

DevBox befindet sich in aktiver Entwicklung. Funktionen, Oberflächen, Datenbankstrukturen, Grafik-Builds, Ordnerstrukturen, Dokumente und Repository-Prozesse können sich ändern, unvollständig sein oder Fehler enthalten. Die Nutzung erfolgt auf eigene Verantwortung.

3. Zulässige Nutzung

DevBox darf zur lokalen Organisation, Dokumentation, Entwicklung und Pflege eigener Softwareprojekte oder von Projekten verwendet werden, für die eine entsprechende Berechtigung besteht. Nutzer sind dafür verantwortlich, dass sie die erforderlichen Rechte an verarbeiteten Inhalten, Daten, Quellcode, Bildern, Schriftdateien, Repositories und externen Diensten besitzen.

4. Externe Programme und Dienste

DevBox kann nach ausdrücklicher Nutzeraktion lokale Programme wie Inkscape, GIMP, Git, Git Credential Manager und Installer starten. Für diese Programme, ihre Lizenzen, Updates, Datenverarbeitung und Nutzungsbedingungen sind die jeweiligen Anbieter oder Rechteinhaber verantwortlich.

Ein Repository-Push, ein Grafik-Build oder ein produktbezogener Prozess wird nur nach expliziter Nutzeraktion vorbereitet oder gestartet. Der Nutzer verantwortet Zielpfade, Repository-URL, Branch, Sichtbarkeit, Commit-Inhalte und die Berechtigung zur Veröffentlichung.

5. Keine Gewährleistung

Soweit gesetzlich zulässig, wird DevBox ohne Zusicherung der Verfügbarkeit, Fehlerfreiheit, Eignung für einen bestimmten Zweck oder vollständigen Kompatibilität bereitgestellt. DevBox ersetzt insbesondere keine Backups, Rechtsprüfung, Sicherheitsprüfung, fachliche Abnahme oder professionelle Freigabe.

6. Schutz des Entwicklungsstands

DevBox verwendet für Export-, Grafik- und Veröffentlichungsoperationen temporäre Arbeitsbereiche. Dennoch bleibt der Nutzer verantwortlich, aktuelle Sicherungen des Projektstands anzulegen und Zielpfade, Datenbankänderungen, Cleanup-Vorgänge, Git-Operationen und Toolstarts vor ihrer Ausführung zu prüfen.

7. Dokumente und rechtliche Inhalte

Mit DevBox erzeugte README-, Lizenz-, Datenschutz- oder Nutzungsbedingungen können technische Vorlagen oder Entwurfsfassungen enthalten. Vor öffentlicher oder kommerzieller Verwendung müssen sie überprüft und gegebenenfalls an das konkrete Produkt, reale Datenflüsse, Vertriebswege, Vertragsbeziehungen und anwendbares Recht angepasst werden.

8. Haftungsbeschränkung

Soweit gesetzlich zulässig, haftet der Autor nicht für mittelbare Schäden, Datenverlust, entgangene Vorteile, Fehlkonfigurationen, fehlerhafte Veröffentlichungen, fehlerhafte Asset-Builds oder Folgen der Nutzung externer Programme und Dienste. Zwingende gesetzliche Ansprüche bleiben unberührt.

9. Änderungen

Diese Entwurfsfassung kann mit dem Entwicklungsstand von DevBox angepasst werden.

3. Lizenzbezug

Lizenz: Zero-Clause BSD License 0BSD

Die vollständigen Lizenzbedingungen sind in der zugehörigen Lizenzdatei hinterlegt.

4. Projektbezogene Hinweise

Zweck: DevBox soll wiederkehrende und fehleranfällige Entwicklungsarbeit in nachvollziehbare lokale Werkzeuge überführen. Dazu gehören insbesondere die Pflege von Produkt- und Herstellerdaten, strukturierte Dokumentation, vorbereitete Veröffentlichungsschritte, die Integration lokaler Kreativprogramme sowie die kontrollierte Ausführung produktbezogener Entwicklungsfunktionen.

Die Anwendung schafft eine gemeinsame Arbeitsgrundlage für Produkte der CYXnTrol Development Platform. Statt Abläufe nur als Erinnerung, Ordnerkonvention oder Sammlung einzelner Konsolenbefehle zu halten, werden sie als Daten, Skripte, GUI-Funktionen und überprüfbare Prozessketten festgehalten.

Für deskNode dient DevBox zusätzlich als Entwicklungs- und Konfigurationsoberfläche für Produktversionen, UX-Themes, Gerätekategorien, Verbrauchersymbole und vorbereitete Grafikpakete. Die spätere deskNode-Laufzeit soll daraus reproduzierbare Daten und bereits gerenderte Assets erhalten.

Kontext: Die CYXnTrol Development Platform entstand aus praktischer Entwicklungsarbeit mit vielen kleinen Werkzeugen, Datenständen, Grafikdateien, Dokumenten und Testabläufen. Mit wachsender Zahl von Produkten und Funktionen reicht es nicht mehr aus, Informationen nur in einzelnen Dateien oder im Gedächtnis zu halten. Benötigt werden eine stabile Projektwurzel, nachvollziehbare Datenquellen, wiederholbare temporäre Arbeitsbereiche und klar abgegrenzte Produktmodule.

DevBox ist die Antwort auf diesen Bedarf. Es bildet eine lokale Entwicklungszentrale, in der Hersteller- und Produktdaten, Dokumentation, globale Strukturregeln, externe Werkzeuge, Repository-Vorbereitung und spezielle Produktfunktionen zusammengeführt werden.

deskNode ist das erste Produkt, das als eigener aktiver Bereich in DevBox integriert wird. Seine Aufgabe ist die Verwaltung und Visualisierung von Smart-Plugs und angeschlossenen Verbrauchern. Für diese Verbraucher werden nicht nur Namen, sondern auch Kategorien, stabile technische IDs, PNG-Quellen, Theme-Daten und vorberechnete Grafikzustände in einen reproduzierbaren Ablauf gebracht. Damit müssen Symbolvarianten nicht mehr manuell für jede Kombination aus Theme und Zustand erstellt werden.

Repository-Hinweis: Das DevBox-Repository repräsentiert die CYXnTrol Development Platform selbst, nicht ein fertiges Endnutzerprodukt. Es dient als Entwicklungsstand, transparente Arbeitsprobe und Grundlage für die Pflege der Plattform.

Der veröffentlichte Stand wird nicht direkt aus dem produktiven Projektroot kopiert. Für DevBox wird ein gesonderter bereinigter root_dir-Stand erzeugt. Er enthält vorgesehenen Quellcode, Dokumente und Assets, während lokale Laufzeitdaten, temporäre Reste,

Installer, unnötige Build-Ausgaben und nicht veröffentlichte Daten außerhalb des Veröffentlichungspakets bleiben.

Der temporäre Veröffentlichungsroot behandelt applications als bewusstes Auslieferungsfenster: Der Bereich wird zunächst bereinigt. Danach wird applications/deskNode/resources/scripts einschließlich enthaltener Dateien in den Veröffentlichungsstand kopiert. __pycache__-Ordner sowie .pyc- und .pyo-Dateien bleiben ausgeschlossen. Der echte lokale Produktordner wird dadurch nicht verändert.

DevBox wird in einem KI-gestützten, iterativen Workflow entwickelt. Anforderungen, Architektur, Datenmodelle, UX-Entscheidungen, Testfälle und Abnahmen werden durch den Projektverantwortlichen gesteuert. Die Implementierung entsteht mit KI-gestützten Entwicklungswerkzeugen und wird gegen die definierten Funktionsanforderungen geprüft.

Copyright (c) 2026 Markus Walloner